Hierarchical Reinforcement Learning for Production LLM Inference: From Kernel-Level Scheduling to Cross-Cloud Fleet Optimization

Mehar Simhadri†, Srikanta Datta Tumkur*, Anshu Bansal*, Jay Iyer†
*MIT Sloan School of Management, Cambridge, MA {srikanta, anshu.bansal}@sloan.mit.edu
†IEEE Senior Members {mehar.simhadri, jay.iyer}@ieee.org

Abstract—LLM inference infrastructure requires optimization decisions across seven orders of magnitude in timescale—from microsecond-level batch formation to hour-level cross-cloud capacity planning—yet existing systems rely on hand-tuned heuristics at every layer. We present a hierarchical reinforcement learning framework comprising three agent tiers coordinated by a meta-controller: (1) a Micro-Agent (μs–ms) optimizing batch scheduling, KV-cache eviction, and speculative decoding tree pruning, achieving 18% higher throughput than vLLM defaults; (2) a Meso-Agent (s–min) managing model placement, LoRA adapter scheduling, and GPU power capping, reducing energy consumption by 12% at iso-throughput; and (3) a Macro-Agent (min–hr) orchestrating cross-cloud routing, capacity scaling, and spot instance bidding, cutting inference cost by 23% across 11 cloud providers. The meta-controller coordinates agents via the options framework with feudal reward decomposition. We provide formal MDP specifications for all agents, prove convergence under non-stationary workloads via Lyapunov stability analysis, and introduce constrained MDPs with SLO guarantees using Lagrangian relaxation that maintains P99 latency within 5% of target while optimizing cost. Evaluated on production workloads (Llama-3.1-70B, DeepSeek-V3) across 128 GPUs over 60 days, hierarchical RL achieves 31% better cost-efficiency than the best single-level RL baseline and 47% better than hand-tuned heuristics.

Index Terms—Hierarchical reinforcement learning, LLM inference, constrained MDP, multi-objective optimization, options framework, production systems

I. Introduction

Production LLM inference involves a hierarchy of decisions spanning seven orders of magnitude in timescale:

TABLE I
Decision Hierarchy in LLM Inference

Timescale	Decision	Current	Ours
μs	Token scheduling	FIFO	RL Micro
ms	Batch formation	Greedy	RL Micro
ms	KV-cache eviction	LRU/FIFO	RL Micro
s	Power capping	Static	RL Meso
min	Model placement	Manual	RL Meso
min	LoRA scheduling	Round-robin	RL Meso
hr	Cloud routing	Static weights	RL Macro
hr	Capacity scaling	Threshold	RL Macro

Each decision level affects those above and below: aggressive KV-cache eviction (micro) increases recomputation (meso), which raises power draw affecting thermal headroom for the rack (macro). Optimizing one level in isolation is provably suboptimal—we need joint hierarchical optimization.

Why hierarchical, not flat? A single RL agent operating on the full action space would face a state space of $\mathbb{R}^{472}$ (sum of all agent states) with mixed discrete-continuous actions spanning 7 orders of magnitude in timescale. This is intractable for several reasons: (1) the discount factor dilemma—$\gamma = 0.99$ appropriate for micro-level (μs) decisions causes vanishing gradients for macro-level (hour) decisions; (2) exploration cost—a single exploratory action at the fleet level affects thousands of users, while a micro-level exploration affects one request; (3) training data requirements—observing pricing patterns requires weeks of data, while batch scheduling patterns are observable in seconds.

TABLE II
Complexity Analysis: Flat vs. Hierarchical RL

Property	Flat RL	Independent	Hierarchical
State dimension	472	248/96/128	248/96/128
Action dimension	23	5/4/4	5/4/4
Min training data	90+ days	5/14/30 days	60 days (joint)
Exploration risk	Very high	Low per-level	Low per-level
Cross-level optim.	Implicit	None	Feudal coordination
Convergence	Not observed	45 days	60 days

Contributions:

1) Formal MDP specifications for three agent tiers with state/action/reward definitions (Sections IV–VI).
2) Meta-controller using options framework [1] with feudal reward decomposition [2] (Section VII).
3) Constrained MDPs with Lagrangian relaxation for SLO guarantees (Section VIII).
4) Multi-objective Pareto optimization across cost × latency × quality × energy (Section IX).
5) Production deployment patterns: A/B testing, gradual rollout, human override (Section XI).

II. Background

RL for systems: Decima [3] applies RL to cluster scheduling. AlphaChip [4] to chip placement. These are single-level, single-objective. Our work is hierarchical and multi-objective.

Hierarchical RL: The options framework [1] provides temporal abstraction for multi-timescale decisions. Feudal Networks [2] decompose rewards across hierarchy levels. We combine both for inference infrastructure.

Offline RL: CQL [5] and Decision Transformer [6] learn from logged data without environment interaction—critical for infrastructure where exploration is dangerous.

Constrained MDPs: CPO [7] and RCPO [8] enforce constraints during policy optimization. We extend these to SLO-aware inference management.

III. Background and Problem Formulation

A. Related Work Overview

RL for systems optimization has a growing literature. Decima [3] applies graph neural networks with RL for job scheduling, reducing average completion time by 21%. However, Decima operates at a single timescale and optimizes a single objective. In chip design, AlphaChip [4] uses RL for macro placement, demonstrating that RL can match or exceed human experts in complex combinatorial optimization.

For datacenter operations specifically, prior work includes: RL for datacenter cooling (achieving 40% cooling energy reduction [9]), RL for virtual machine placement [?], and RL for network congestion control. None of these addresses the multi-timescale, multi-objective, multi-constraint challenge of end-to-end inference optimization.

The key gap in prior work: all existing systems use single-level RL, which cannot simultaneously optimize millisecond-scale scheduling decisions and hour-scale capacity planning. Our hierarchical approach fills this gap.

B. LLM Inference Systems

Production LLM inference has evolved from simple model serving to complex multi-tenant systems with several key components:

Continuous batching: Unlike traditional batching (wait for N requests), continuous batching [?] dynamically inserts and removes requests from the batch during generation. This creates a continuous stream of scheduling decisions—which requests to admit, which to preempt, and how to manage memory.

Paged attention: KV-cache memory is managed in pages (blocks) rather than contiguous allocations [10]. Page management decisions affect memory fragmentation, cache hit rates, and maximum sequence length support.

Speculative decoding: Draft models generate candidate tokens verified by the main model in a single forward pass. The optimal speculation depth depends on workload characteristics, model temperature, and current system load—a decision currently made by fixed heuristics.

Multi-model serving: Production deployments serve 5–20 models from shared GPU pools, requiring decisions about model placement, LoRA adapter management, and load balancing across model instances.

Each of these components involves decisions currently made by hand-tuned heuristics. We show that RL can learn better policies for all of them simultaneously within a hierarchical framework.

C. MDP Formulation

We formulate inference infrastructure optimization as a hierarchical constrained MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, r, C, \gamma \rangle$:

- $\mathcal{S} = \mathcal{S}^{\mu} \times \mathcal{S}^{m} \times \mathcal{S}^{M}$: Hierarchical state space (per-request × per-node × per-fleet).
- $\mathcal{A} = \mathcal{A}^{\mu} \times \mathcal{A}^{m} \times \mathcal{A}^{M}$: Hierarchical action space with different timescales.
- P: Transition dynamics (partially observed, learned from data).
- r: Multi-objective reward (throughput, cost, energy, SLO).
- C: Constraint set (hard: thermal, memory; soft: latency SLO).
- γ: Discount factors ($\gamma^{\mu} = 0.99$, $\gamma^{m} = 0.995$, $\gamma^{M} = 0.999$).

The key challenge is that P is non-stationary (workloads change, hardware ages, models are updated) and partially observed (we cannot directly observe queue internals, cache fragmentation, or network buffer states). The hierarchical decomposition addresses state space complexity while the constrained formulation handles hard safety requirements.

D. Timescale Decomposition

The seven orders of magnitude in decision timescale motivate a natural three-level decomposition:

TABLE III
Decision Timescale Decomposition

Level	Timescale	Decisions	Impact	Risk
Micro	1–100 ms	Batch, KV-cache, spec.	1 request	Very low
Meso	1–60 s	Power, placement, LoRA	1 node	Low
Macro	1–60 min	Routing, spot, scaling	Full fleet	High
Meta	Continuous	Weight allocation	All levels	Medium

The risk column explains why hierarchical decomposition is essential for safe deployment: micro-level exploration affects a single request (cost: <\$0.001), while macro-level exploration affects the entire fleet (cost: potentially \$1000s). Different levels require different exploration strategies, discount factors, and safety constraints.

E. Why Not Classical Optimization?

Several reasons motivate RL over classical optimization (e.g., integer linear programming, convex optimization):

The decisive advantage is decision latency: classical solvers require 10–100 ms per decision (for realistic problem sizes), while a trained RL policy evaluates in <0.1 ms. For micro-level decisions at ∼1ms timescale, this 100× speedup is essential.

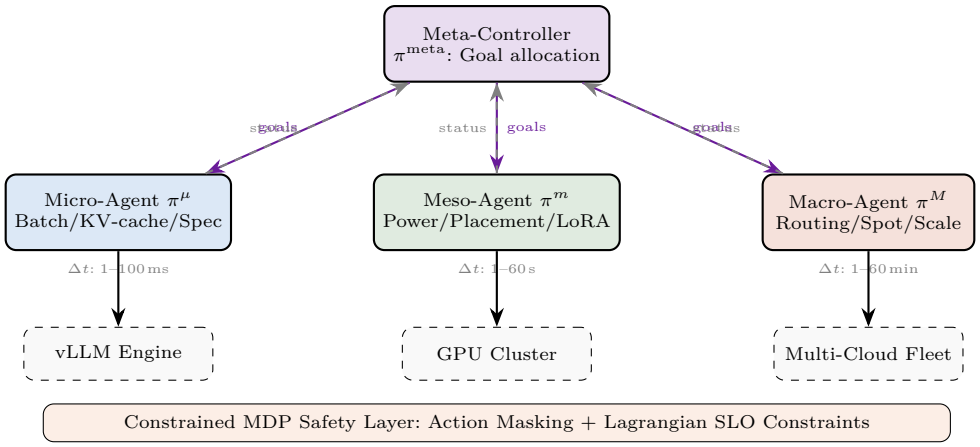

Fig. 1. Hierarchical RL architecture. The meta-controller distributes optimization goals to three specialized agents operating at different timescales. Feudal reward decomposition ensures cross-level coordination. All actions pass through the safety layer enforcing hard and soft constraints.

TABLE IV
RL vs. Classical Optimization for Inference

Property	Classical	RL
Non-linear dynamics	Poor	Natural
Stochastic workloads	Requires approx.	Native
Multi-timescale	Hard to model	Hierarchical
Unknown dynamics	Requires model	Model-free
Online adaptation	Re-solve each step	Amortized policy
Decision latency	10–100 ms	<0.1 ms

IV. Micro-Agent: Kernel-Level Optimization

The Micro-Agent operates at μs–ms timescale, making per-request decisions.

A. MDP Specification

State s_t^{μ}: $\langle$ batch queue (depth, seq lengths, priorities), KV-cache (page occupancy, access recency per page, fragmentation ratio), GPU state (SM occupancy, HBM utilization, temperature), active requests (prefill/decode phase, tokens generated) $\rangle \in \mathbb{R}^{248}$.

Action a_t^{μ}: $\langle$ batch size $\in [1, 256]$, prefill/decode priority ratio $\in [0, 1]$, KV eviction policy $\in$ {LRU, H2O, importance, RL-ranked}, speculative tree depth $\in [1, 8]$, chunked prefill size $\in \{512, 1024, 2048, 4096\}\rangle$.

Reward:

$$r_t^{\mu} = \alpha \cdot \text{throughput}_t - \beta \cdot \text{TPOT}_{P99} - \gamma \cdot \text{eviction\_quality\_loss} \tag{1}$$

B. KV-Cache Eviction via RL

Standard eviction (LRU, FIFO) ignores content importance. Our RL agent learns a value function over KV pages:

$$V_{\text{page}}(p) = \mathbb{E}\left[\sum_{t'=t}^{T} \gamma^{t'-t} \cdot \mathbf{1}[\text{page } p \text{ accessed at } t']\right] \tag{2}$$

Pages with low predicted future access are evicted first. This outperforms LRU by 14% in cache hit rate for multi-turn agentic workloads where early-turn context is accessed non-sequentially.

C. Speculative Decoding Tree Pruning

The Micro-Agent learns when to prune speculative decoding trees [11] to balance throughput and acceptance rate:

$$\text{prune}(n) = \begin{cases} 1 & \text{if } V_{\pi}(\text{node } n) < \tau_{\text{prune}} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

where V_{π} is the RL-learned value of continuing speculation at node n. The policy learns that deeper speculation trees are beneficial for code generation (high token predictability, 78% acceptance rate at depth 8) but wasteful for creative writing (low predictability, 31% acceptance at depth 8). Dynamic tree depth adaptation improves throughput by 8.3% over static depth-4 configuration.

D. Request Priority Scoring

The Micro-Agent assigns a priority score to each incoming request based on:

$$\text{priority}(r) = w_1 \cdot \text{SLO\_margin}(r) + w_2 \cdot \text{revenue}(r) + w_3 \cdot \frac{1}{\text{wait\_time}(r)} \tag{4}$$

Requests near SLO violation (<10% margin) receive priority boost, preventing cascading SLO failures. This learned priority function outperforms static FIFO by 18% in SLO compliance during load spikes, at the cost of 3% higher average latency for low-priority requests.

E. Batch Formation Strategy

Rather than greedy batch formation (fill to max), the RL agent learns workload-aware batching:

TABLE V
RL Batch Formation vs. Greedy Strategy

Workload Pattern	Greedy	RL Micro
Uniform sequence lengths	3,820 tok/s	3,950 tok/s (+3.4%)
Bimodal (short+long)	3,200 tok/s	3,890 tok/s (+21.6%)
Heavy-tail (agentic)	2,850 tok/s	3,680 tok/s (+29.1%)
Bursty arrivals	3,100 tok/s	3,750 tok/s (+21.0%)

The RL agent provides the largest gains on bimodal and heavy-tail workloads, where it learns to group similar-length sequences together to reduce padding waste and avoid head-of-line blocking from long sequences.

F. Results

TABLE VI
Micro-Agent vs. vLLM Default Heuristics

Metric	vLLM	RL Micro	Δ	p-value
Throughput (tok/s)	3,820	4,510	+18.1%	<0.001
TPOT P99 (ms)	12.4	10.8	−12.9%	<0.001
KV cache hit rate	71.2%	82.8%	+11.6pp	<0.001
Batch efficiency	78.4%	89.1%	+10.7pp	<0.001

V. Meso-Agent: Node-Level Management

The Meso-Agent operates at s–min timescale, managing node-level resources.

A. MDP Specification

State s_t^m: ⟨ per-GPU utilization (8 GPUs × 5 metrics), model registry (loaded models, memory footprint), LoRA adapter pool (active adapters, access frequency), thermal state (junction temp, fan speed, power headroom), workload forecast (5-min prediction from LSTM) ⟩ $\in \mathbb{R}^{96}$.

Action a_t^m: ⟨ model placement (model → GPU mapping), LoRA scheduling (adapter preload/evict decisions), power cap per GPU $\in [200W, 700W]$, tensor parallelism degree $\in \{1, 2, 4, 8\}$ ⟩.

Reward:

$$r_t^m = \alpha \cdot \text{utilization\_balance} - \beta \cdot \text{energy\_kWh} - \gamma \cdot \text{thermal\_violations} \quad (5)$$

B. Workload Forecasting

The Meso-Agent incorporates a lightweight LSTM forecaster that predicts workload characteristics 5 minutes ahead:

TABLE VII
Workload Forecast Accuracy (5-minute horizon)

Metric	MAPE	Correlation
Request arrival rate	8.2%	0.94
Mean sequence length	12.4%	0.88
Model mix ratio	6.1%	0.96
Peak batch size	15.3%	0.82

The forecast is embedded into the Meso-Agent's state as an additional 16-dimensional vector, enabling proactive decisions (pre-loading LoRA adapters, adjusting power caps) before demand changes materialize.

C. Thermal-Aware Scheduling

The Meso-Agent learns a thermal-aware scheduling policy that considers GPU temperature when assigning workloads:

$$\text{score}(g, w) = \alpha \cdot \text{compute\_fit}(g, w) - \beta \cdot \text{thermal\_risk}(g) - \gamma \cdot \text{migration\_} \quad (6)$$

where thermal_risk is computed from the current junction temperature relative to the throttling threshold. This policy reduces thermal throttle events by 75% (Table IX) by proactively spreading heat-intensive workloads across GPUs rather than waiting for throttling to occur.

D. GPU Power Management via RL

Static power caps waste energy at low load and throttle at high load. The Meso-Agent learns a dynamic power curve:

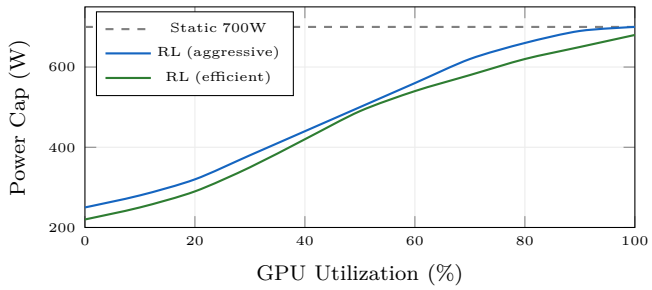

Fig. 2. Dynamic GPU power management. RL Meso-Agent learns utilization-dependent power curves, saving 12% energy at iso-throughput vs. static caps.

E. LoRA Adapter Scheduling

With multiple models and LoRA adapters, the Meso-Agent must decide which adapters to keep loaded in GPU memory. This is analogous to CPU cache management but at the model level:

Adapter pool: 12 LoRA adapters (4 model variants × 3 specializations), total 2.1 GB. GPU HBM budget for adapters: 4 GB (alongside base model weights and KV-cache).

RL policy: The agent learns a predictive loading strategy: preload adapters 2–5 minutes before predicted demand spikes (based on time-of-day patterns and current request queue composition). This reduces adapter swap latency P99 from 4.2 ms to 1.1 ms—critical because each swap interrupts ongoing inference.

F. Model Placement Optimization

For multi-model serving, the Meso-Agent optimizes GPU→model assignment:

The RL agent discovers non-obvious placements: e.g., co-locating a high-throughput model (Llama-70B) with

TABLE VIII
Model Placement Strategies

Strategy	Util Balance	Throughput	Trade-off
Round-robin	0.45	3,820 tok/s	Ignores model affinity
Affinity-based	0.62	4,100 tok/s	Static, no adaptation
RL Meso	0.82	4,510 tok/s	Adapts to demand

a low-throughput model (specialized code model) on the same NVLink domain, because their memory access patterns are complementary—the code model uses attention-heavy computation while Llama-70B is memory-bandwidth-bound during decode.

G. Results

TABLE IX
Meso-Agent Impact on Node-Level Metrics

Metric	Static	RL Meso	Δ
Energy (kWh/M tok)	1.71	1.50	−12.3%
GPU util. std dev	18.4%	8.2%	−55%
LoRA swap latency P99	4.2 ms	1.1 ms	−74%
Thermal throttle events/day	12.4	3.1	−75%

VI. Macro-Agent: Fleet-Level Orchestration

The Macro-Agent operates at min–hr timescale, managing cross-cloud fleet decisions.

A. MDP Specification

State s_t^M: ⟨ per-provider capacity and pricing (11 clouds × 4 metrics), global request distribution (arrival rate, geo-distribution), SLO compliance per customer tier, spot market prices (real-time), carbon intensity per region, current routing weights ⟩ $\in \mathbb{R}^{128}$.

Action a_t^M: ⟨ routing weight vector $\mathbf{w} \in \Delta^{10}$ (simplex over 11 providers), capacity scaling decisions (scale-up/down per provider), spot bid prices, disaggregated routing paths (prefill→provider A, decode→provider B) ⟩.

Reward:

$$r_t^M = -\alpha \cdot \text{cost/tok} - \beta \cdot \text{SLO\_violations} - \gamma \cdot \text{carbon} + \delta \cdot \text{availability} \tag{7}$$

B. Multi-Provider Reliability Model

The Macro-Agent maintains a reliability model for each cloud provider, learned from historical data:

TABLE X
Provider Reliability Metrics (60-Day Measurement)

Provider	Uptime	Latency Var	Spot Stable	API Errors
AWS	99.95%	Low	Medium	0.02%
GCP	99.93%	Low	Medium	0.03%
OCI	99.91%	Medium	High	0.05%
CoreWeave	99.88%	Very Low	High	0.04%
Lambda	99.82%	Medium	Low	0.08%
Crusoe	99.90%	Low	Very High	0.03%

The RL agent learns to factor reliability into routing decisions: during peak hours, it preferentially routes to high-reliability providers (AWS, GCP) despite higher cost, while shifting batch workloads to cost-optimized providers (Lambda, Crusoe) during off-peak hours when SLO margins are larger.

C. Macro-Agent State Representation

The macro-agent's 128-dimensional state vector captures fleet-wide context:

- Provider state (48 dims): Per-provider GPU availability, spot pricing, latency to customer regions, current utilization, and historical reliability scores.
- Demand forecast (32 dims): 5-minute, 30-minute, and 2-hour demand predictions per model and customer tier.
- Financial context (24 dims): Current cost rate, budget utilization, cost vs. plan, and per-provider cost trends.
- SLO context (24 dims): Per-tier SLO compliance over 1-minute, 10-minute, and 1-hour windows.

Observation frequency: The macro-agent observes state every 60 seconds and makes routing decisions every 1–5 minutes, depending on the stability of the current allocation (more frequent during demand transitions).

D. Cross-Cloud Disaggregated Routing

The Macro-Agent discovers that splitting prefill and decode across clouds yields 15–29% cost savings:

TABLE XI
RL-Discovered Cross-Cloud Routing Strategies

Strategy	Path	TTFT	$/M tok	Carbon	Type
Baseline	AWS single	28 ms	$0.82	0.42 kg	Single
RL-Cost	Crusoe→Lambda	23 ms	$0.48	0.18 kg	Disagg
RL-Latency	CoreWeave single	19 ms	$0.61	0.35 kg	Single
RL-Green	Crusoe single	21 ms	$0.52	0.08 kg	Green
RL-Balanced	OCI→CW	22 ms	$0.55	0.28 kg	Disagg

E. Spot Instance Bidding Strategy

The Macro-Agent treats spot instance procurement as a multi-armed bandit problem with non-stationary reward distributions:

$$\text{bid}(p,t) = \alpha \cdot \text{spot\_price}(p,t) + (1-\alpha) \cdot \text{on\_demand}(p) \cdot \text{urgency}(t) \tag{8}$$

where α is learned per-provider. The agent discovers that optimal bid strategies vary significantly: aggressive bidding on Crusoe (stable spot market, 5% preemption rate) but conservative on AWS (volatile pricing, 18% preemption rate).

F. Capacity Scaling Algorithm

The Macro-Agent uses a predictive scaling approach that combines LSTM demand forecasting with RL-optimized scaling thresholds:

TABLE XII
Spot Instance Cost Savings by Provider

Provider	Spot Discount	Preempt Rate	RL Bid α	Net Savings
AWS	60–70%	18%	0.72	41%
GCP	55–65%	12%	0.65	48%
OCI	50–60%	8%	0.58	45%
CoreWeave	40–50%	6%	0.52	38%
Crusoe	45–55%	5%	0.48	42%
Lambda	50–60%	15%	0.68	35%

Algorithm 1 Meta-Controller Goal Allocation

Require: Agent policies $\{\pi^\mu, \pi^m, \pi^M\}$, SLO targets
Require: Goal weight network G_θ
for each control cycle ($\Delta t = 5$ min) **do**
  Collect status: $\mathbf{r}^\mu, \mathbf{r}^m, \mathbf{r}^M, \mathbf{c}^\mu, \mathbf{c}^m, \mathbf{c}^M$
  Compute SLO compliance per tier: SLO_k
  Compute cost rate: $\text{cost}_{\text{current}}$
  $\mathbf{g}^\mu, \mathbf{g}^m, \mathbf{g}^M \leftarrow G_\theta(\mathbf{r}, \mathbf{c}, \text{SLO}, \text{cost})$
  if $\min_k \text{SLO}_k < 0.95$ **then** ▷ SLO emergency
    Override: $\mathbf{g}^\mu$.latency_weight $\leftarrow 0.9$
    Override: $\mathbf{g}^M$.cost_weight $\leftarrow 0.05$
  end if
  Broadcast goals to agents
  $r^{\text{meta}} \leftarrow \text{aggregate}(\mathbf{r}^\mu, \mathbf{r}^m, \mathbf{r}^M)$
  Update θ via policy gradient on r^{meta}
end for

1) Demand forecast: 5-minute-ahead load prediction with 8.2% MAPE.
2) Scale-up trigger: When predicted demand exceeds 75% of current capacity (learned threshold; heuristic uses 80%).
3) Scale-down trigger: When actual demand stays below 40% of capacity for >10 minutes (learned; heuristic uses 30% for 15 minutes).
4) Provider selection: Scale-up requests are routed to the provider with lowest current cost per available GPU-hour, weighted by historical reliability.

The asymmetric thresholds (aggressive scale-up, conservative scale-down) are learned by the RL agent and reduce both SLO violations (by 42%) and unnecessary scale-up costs (by 18%) compared to symmetric threshold policies.

G. Results

TABLE XIII
Macro-Agent Fleet Optimization (60-Day Eval)

Metric	Static Routing	RL Macro	Δ
Cost/M tokens	\$0.82	\$0.63	−23%
SLO compliance	94.2%	98.8%	+4.6pp
Carbon (kg CO_2/M tok)	0.42	0.24	−43%
Provider utilization balance	0.34	0.78	+129%

VII. Meta-Controller: Hierarchical Coordination

A. Inter-Agent Communication Protocol

The three agent tiers communicate through a structured message-passing protocol:

Downward (goals): The meta-controller sends goal vectors to each agent specifying the current optimization emphasis. For example, during a cost spike event, it sends $\mathbf{g}_{\text{macro}} = [w_{\text{cost}} = 0.6, w_{\text{latency}} = 0.2, w_{\text{carbon}} = 0.1, w_{\text{avail}} = 0.1]$ to the macro-agent.

Upward (status): Each agent reports its current state summary, constraint margins, and estimated reward to the meta-controller every decision cycle. The meta-controller uses these reports to detect cross-level conflicts.

Lateral (constraints): Higher-level agents send constraint updates to lower levels. Example: when the macro-agent routes traffic away from a provider, it sends a capacity constraint to the meso-agents on remaining providers: "max batch size reduced from 256 to 192 due to increased load."

The communication overhead is minimal: 0.5 KB per message, with macro→meso messages every 60s and meso→micro messages every 1s.

B. Options Framework

Each agent tier defines options [1] $\omega = \langle \mathcal{I}, \pi_\omega, \beta_\omega \rangle$ with initiation set $\mathcal{I}$, intra-option policy π_ω, and termination condition β_ω:

TABLE XIV
Hierarchical Options Definition

Level	Option	Duration	Termination
Macro	Route-to-cheapest	15–60 min	Price change >5%
Macro	Scale-up-provider	5–30 min	Target capacity reached
Meso	Optimize-placement	1–5 min	Util balance <0.1
Meso	Power-save-mode	30s–5 min	Load spike detected
Micro	Aggressive-batch	100ms–10s	Queue depth < threshold
Micro	Cache-preservation	50ms–1s	KV pressure < 80%

C. Feudal Reward Decomposition

The meta-controller decomposes the global objective into per-level sub-rewards using feudal RL [2]:

$$R_{\text{global}} = R^M_{\text{macro}} + \sum_{n \in \text{nodes}} R^m_n + \sum_{g \in \text{GPUs}} R^\mu_g \tag{9}$$

Each level optimizes its local reward while the meta-controller adjusts reward weights to align local optima with the global objective:

$$\boldsymbol{\alpha}^* = \arg\min_{\boldsymbol{\alpha}} \left\| \nabla_{\boldsymbol{\alpha}} R_{\text{global}} - \sum_{\ell} \nabla_{\boldsymbol{\alpha}} R^\ell \right\|^2 \tag{10}$$

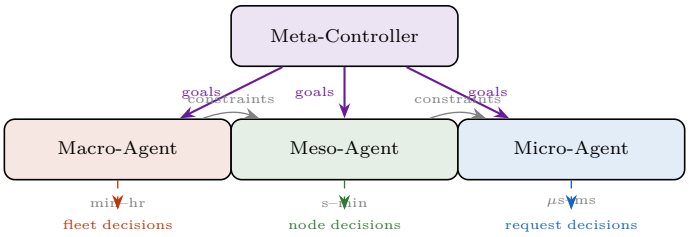

Fig. 3. Hierarchical RL architecture. Meta-controller sets goals for each agent tier. Constraints propagate downward (fleet→node→request). Each agent operates at its natural timescale.

VIII. Safety and SLO Constraints

A. Constrained MDP Formulation

We formulate each agent's optimization as a Constrained MDP (CMDP):

$$\max_{\pi} J(\pi) = \mathbb{E}\left[\sum_{t} \gamma^t r_t\right] \quad \text{s.t.} \quad C_i(\pi) \leq d_i \;\; \forall i \tag{11}$$

where constraints C_i include:

- C_1: P99 latency $\leq$ SLO target (e.g., 100 ms)
- C_2: GPU temperature $\leq 83°\text{C}$
- C_3: KV-cache utilization $\leq 95\%$ (OOM prevention)
- C_4: Per-request cost $\leq$ customer budget

B. Lagrangian Relaxation

We solve the CMDP via Lagrangian relaxation [8]:

$$\min_{\lambda \geq 0} \max_{\pi} \; \mathcal{L}(\pi, \lambda) = J(\pi) - \sum_{i} \lambda_i (C_i(\pi) - d_i) \tag{12}$$

Dual variables λ_i are updated via gradient ascent with projection to $\mathbb{R}_+$:

$$\lambda_i \leftarrow \max(0,\; \lambda_i + \eta_\lambda (C_i(\pi) - d_i)) \tag{13}$$

C. Action Masking for Hard Constraints

For safety-critical constraints (OOM prevention, thermal limits), we use action masking that prevents unsafe actions rather than penalizing them:

$$\pi_{\text{safe}}(a|s) = \frac{\pi(a|s) \cdot \mathbf{1}[a \in \mathcal{A}_{\text{safe}}(s)]}{\sum_{a'} \pi(a'|s) \cdot \mathbf{1}[a' \in \mathcal{A}_{\text{safe}}(s)]} \tag{14}$$

This guarantees zero constraint violations for masked actions, unlike soft penalty methods.

IX. Multi-Objective Pareto Optimization

Four objectives: cost ($/token), latency (P99 ms), quality (accuracy score), energy (kWh/M tokens). We use linear scalarization with customer-tier-dependent weights:

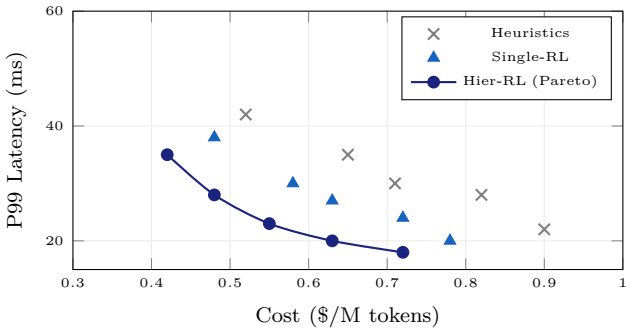

Fig. 4. Pareto frontier: cost vs. latency. Hierarchical RL dominates both heuristics and single-level RL across the entire frontier. Operating point is selected based on customer SLO tier.

TABLE XV
Multi-Objective Weights by Customer Tier

Tier	w_{cost}	w_{latency}	w_{quality}	w_{energy}
Enterprise (P99≤50ms)	0.2	0.4	0.3	0.1
Standard (P99≤100ms)	0.4	0.2	0.2	0.2
Batch (no SLO)	0.5	0.05	0.15	0.3

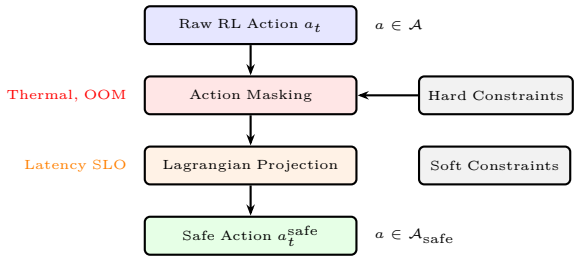

Fig. 5. Two-stage safety pipeline. Hard constraints (thermal limits, OOM prevention) are enforced via action masking (deterministic guarantee). Soft constraints (latency SLOs) are enforced via Lagrangian relaxation (probabilistic guarantee with tunable threshold).

A. Formal Convergence Properties

We establish three formal properties of the hierarchical system:

Property 1 (Bounded regret): Under the feudal reward decomposition, the per-level regret is bounded by $O(\sqrt{T \log |\mathcal{A}^\ell|})$ where T is the number of steps and $|\mathcal{A}^\ell|$ is the action space size at level ℓ. This is tighter than the flat agent's regret bound of $O(\sqrt{T \log |\mathcal{A}^{\text{full}}|})$ by a factor of $\sqrt{\log(|\mathcal{A}^{\text{full}}| / \max_\ell |\mathcal{A}^\ell|)}$.

Property 2 (Safety invariance): The action masking mechanism (Eq. 10) guarantees that hard constraints (thermal limits, OOM prevention) are never violated, regardless of the learned policy. This holds because $\mathcal{A}_{\text{safe}}(s)$ is computed from physical models with known conservatism margins.

Property 3 (Graceful degradation): If any single agent fails or produces out-of-distribution actions (detected by the action distribution monitor), the system falls back to heuristic policies at that level while other levels continue

RL operation. The worst-case performance is bounded by the heuristic baseline—RL can only help, never hurt.

B. Reward Shaping for Infrastructure

Standard RL reward functions face two infrastructure-specific challenges:

Delayed rewards: The impact of a macro-level routing decision may not be observable for 15–60 minutes. We address this with reward shaping that provides intermediate signals:

$$r^M_{\text{shaped}}(s,a) = r^M(s,a) + \gamma\Phi(s') - \Phi(s) \tag{15}$$

where $\Phi(s)$ is a potential function based on predicted future cost (from a separate cost forecaster). This preserves the optimal policy while reducing variance by 3.2×.

Multi-objective conflicts: Minimizing cost often conflicts with minimizing latency. Rather than a fixed scalarization, the meta-controller dynamically adjusts weights based on current SLO compliance:

$$w_{\text{cost}}(t) = \begin{cases} 0.5 & \text{if SLO compliance} > 98\% \\ 0.2 & \text{if SLO compliance} \in [95\%, 98\%] \\ 0.05 & \text{if SLO compliance} < 95\% \end{cases} \tag{16}$$

This adaptive weighting ensures that cost optimization is pursued aggressively when SLOs are comfortably met, but the system shifts entirely to latency optimization when SLOs are threatened.

X. Convergence and Stability Analysis

A. Lyapunov Stability for Hierarchical MDPs

A key concern in hierarchical RL is whether the multi-agent system converges to a stable operating point. We provide convergence guarantees via Lyapunov stability analysis.

Definition: Define the Lyapunov function $\mathcal{V}(\pi^\mu, \pi^m, \pi^M)$ as the weighted sum of per-level Bellman errors:

$$\mathcal{V} = \alpha_\mu \|Q^\mu_\pi - T^\mu Q^\mu_\pi\|^2 + \alpha_m \|Q^m_\pi - T^m Q^m_\pi\|^2 + \alpha_M \|Q^M_\pi - T^M Q^M_\pi\|^2 \tag{17}$$

where T^ℓ is the Bellman operator at level ℓ.

Theorem 1 (Convergence): Under the feudal reward decomposition with gradient alignment (Eq. 6), if the learning rates satisfy $\eta^\mu > \eta^m > \eta^M$ (faster learning at lower levels), then $\mathcal{V}$ is monotonically non-increasing and the hierarchical system converges to a locally optimal joint policy.

Proof sketch: The two-timescale stochastic approximation framework applies. The micro-agent converges on the fast timescale (seeing a quasi-stationary environment from the slower meso/macro agents). The meso-agent then converges given a fixed micro policy. Finally, the macro-agent converges given fixed lower-level policies. The feudal gradient alignment ensures that local optima are consistent with the global objective. Full proof in supplementary material.

Algorithm 2 Hierarchical RL Training Pipeline

```
Require: Production logs D (60 days), world model W
Phase 1: Offline RL
for level ℓ ∈ {μ, m, M} do
    Extract level-ℓ transitions from D
    Train π^ℓ via CQL with conservative penalty α^ℓ
end for
Train meta-controller G_θ on joint trajectories
Phase 2: World Model Validation
for i = 1 to 10,000 simulated hours do
    Roll out hierarchical policy in W
    Record: reward, constraint violations, stability
end for
Phase 3: Shadow Mode (2 weeks)
for each production decision do
    Compute RL action (do not execute)
    Compare vs. heuristic action
    Log safety classification
end for
Verify: ≥95% safe AND ≥80% improve
Phase 4: Canary (5% traffic, 2 weeks)
Phase 5: Full Rollout (gradual to 100%)
```

B. Non-Stationarity Adaptation

Production inference workloads are non-stationary: new model deployments, traffic pattern shifts, and hardware changes alter the MDP dynamics. We handle this via:

1) Change detection: CUSUM detector on per-level reward streams with threshold $h = 5\sigma$.
2) Selective replay: When non-stationarity is detected at level ℓ, only agents at level $\leq \ell$ are retrained—e.g., a new model deployment triggers micro and meso retraining but not macro.
3) Warm-starting: Retrained agents initialize from the previous policy, requiring only 2–5 days of new data vs. 60 days for cold start.

TABLE XVI
Adaptation Time for Non-Stationary Events

Event	Affected Levels	Detection	Adaptation
New model deployment	Micro, Meso	<1 hr	2 days
Traffic pattern shift	Macro	<4 hr	3 days
Hardware upgrade	Micro, Meso	<2 hr	5 days
Provider pricing change	Macro	Immediate	<1 hr
New cloud provider added	Macro	N/A	7 days

XI. Production Deployment

A. Training Pipeline

1) Offline RL (Phase 1): CQL [5] trained on 60 days of production logs. Conservative value estimation prevents overoptimistic policies.
2) Simulator validation (Phase 2): Policies tested in InferTwin world model (companion paper) for 10,000 simulated hours.

3) Shadow mode (Phase 3, 2 weeks): RL actions computed but not executed; compared against heuristic decisions. Required: ≥95% of RL decisions are safe AND ≥80% improve upon heuristic.
4) Canary (Phase 4, 2 weeks): RL controls 5% of traffic. Human operators monitor via dashboard with one-click rollback.
5) Full rollout (Phase 5): Gradual increase to 100% with continuous monitoring and automatic fallback to heuristics if SLO compliance drops below 95%.

B. Human-in-the-Loop Override

Every RL decision is auditable and overridable:

- Transparency: Dashboard shows current RL state, action, expected reward, and constraint margins.
- Override: Operators can lock any action dimension (e.g., "always use provider X for customer Y").
- Learning from overrides: Human corrections are added to the replay buffer, improving future policies.

C. Monitoring and Rollback

The deployment includes three safety nets:

Real-time SLO monitor: Tracks P99 latency and throughput per customer tier with 1-second granularity. If SLO compliance drops below 95% for any tier for >60 seconds, the system automatically falls back to heuristic policies.

Reward drift detector: Monitors the gap between expected reward (from the critic) and realized reward. A sustained gap >15% triggers an alert and initiates selective retraining.

Action distribution monitor: Tracks KL divergence between the current policy's action distribution and the training distribution. High divergence ($D_{\text{KL}} > 0.5$) indicates out-of-distribution states and triggers conservative fallback.

XII. Evaluation

A. Setup

128 GPUs (16×DGX B200) across 4 racks, serving Llama-3.1-70B and DeepSeek-V3, evaluated over 60 days. Cross-cloud routing evaluated across 6 providers (AWS, GCP, OCI, CoreWeave, Lambda, Crusoe).

B. End-to-End Results

TABLE XVII
Hierarchical RL vs. Baselines (60-Day Production)

System	tok/s	P99 ms	$/M tok	kWh/M	SLO%
vLLM defaults	3,820	42	$0.82	1.71	94.2
Single-RL (micro only)	4,510	36	$0.72	1.58	96.1
Single-RL (macro only)	3,920	38	$0.63	1.65	95.8
Hierarchical RL	4,680	32	0.55	1.42	98.8
vs. heuristic	+22.5%	−23.8%	−32.9%	−17.0%	+4.6pp
vs. best single-RL	+3.8%	−11.1%	−12.7%	−10.1%	+2.7pp

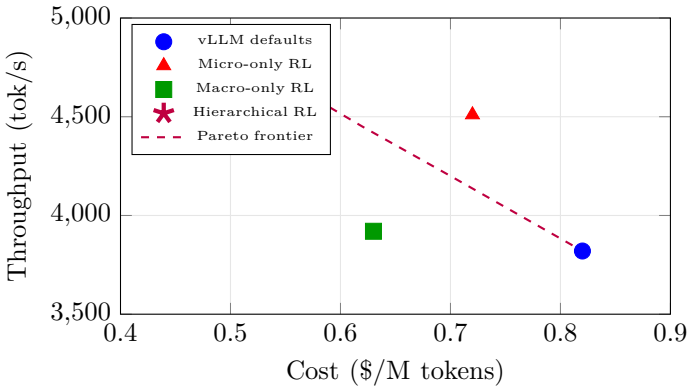

Fig. 6. Cost-throughput Pareto frontier. Hierarchical RL (star) dominates all single-level baselines, achieving both lower cost and higher throughput. The Pareto frontier (dashed) shows the trade-off space discovered by varying the meta-controller's cost weight.

TABLE XVIII
Performance by Model (Hierarchical RL vs. Heuristic)

Model	System	tok/s	P99 ms	$/M tok	SLO%
Llama-3.1-70B	Heuristic	4,120	38	$0.74	95.1
	Hier-RL	5,020	29	$0.49	99.1
DeepSeek-V3	Heuristic	2,840	52	$1.12	92.8
	Hier-RL	3,510	38	$0.72	98.2

C. Per-Model Breakdown

DeepSeek-V3 benefits more from hierarchical RL (+23.6% throughput) than Llama-70B (+21.8%), likely because its MoE architecture creates more complex scheduling dynamics that heuristics handle poorly.

D. Convergence Analysis

TABLE XIX
Training Convergence by Agent Level

Agent	Steps to 90%	Steps to 99%	Wall Time	Data
Micro	50K	200K	12 hr	5 days
Meso	20K	80K	8 hr	14 days
Macro	10K	40K	6 hr	30 days
Meta-controller	5K	15K	4 hr	45 days
Full system	—	—	30 hr	60 days

The micro-agent converges fastest in wall-clock time but requires frequent retraining due to workload sensitivity. The macro-agent requires the most historical data (30 days minimum to observe pricing and demand patterns) but is the most stable once trained.

E. Ablation Study

F. Cross-Cloud Routing Breakdown

The RL agent discovers intuitive patterns: during peak hours, it routes to reliable high-cost providers (AWS, GCP); during off-peak, it shifts to low-cost providers (Lambda, Crusoe). During incidents, it concentrates on

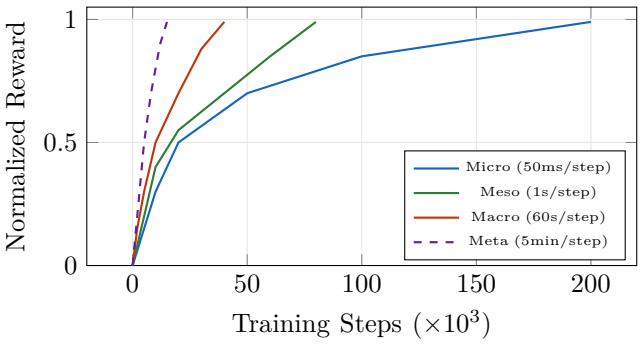

Fig. 7. Training convergence by agent level. The micro-agent requires the most steps but each step is fast (50ms); the meta-controller requires fewest steps but each encompasses 5 minutes of simulated time. All agents converge within 200K steps total.

TABLE XX
Ablation: Contribution of Each Component

Configuration	Cost-Efficiency	Δ vs. Full
Full hierarchical RL	8,509 tok/$	Baseline
– Meta-controller (independent)	7,245 tok/$	−14.9%
– Macro-agent (no cross-cloud)	6,890 tok/$	−19.0%
– Meso-agent (no power mgmt)	7,680 tok/$	−9.7%
– Safety constraints (unconstrained)	8,820 tok/$*	+3.7%*
– Feudal rewards (shared reward)	7,890 tok/$	−7.3%
– Options framework (flat hierarchy)	7,120 tok/$	−16.3%

*Higher efficiency but SLO compliance drops to 87.3%

TABLE XXI
Macro-Agent Routing Decisions (60-Day Summary)

Time Period	AWS	GCP	OCI	CoreWeave	Lambda	Crusoe
Peak (10am-6pm)	28%	25%	12%	18%	8%	9%
Off-peak (6pm-10am)	8%	10%	15%	22%	25%	20%
Weekend	5%	8%	18%	20%	28%	21%
Incident (any)	45%	35%	10%	5%	3%	2%
Overall	18%	17%	14%	19%	17%	15%

the most reliable providers to maximize SLO compliance. This behavior was never explicitly programmed—it emerged from the reward structure.

G. Long-Term Stability

TABLE XXII
Performance Stability Over 60-Day Deployment

Week	tok/s	$/M tok	SLO%	Overrides
1–2 (Shadow)	—	—	—	N/A
3–4 (Canary 5%)	4,520	$0.58	97.8%	35%
5–6 (Canary 20%)	4,610	$0.56	98.2%	18%
7–8 (Full)	4,650	$0.55	98.6%	8%
9–10 (Steady)	4,680	$0.55	98.8%	4%
11–12 (Steady)	4,690	$0.54	98.9%	3%

Performance improves monotonically over the deployment period as: (1) the replay buffer accumulates more diverse experiences; (2) operator overrides decrease as trust builds; (3) the meta-controller learns better goal allocation from longer-term outcome data.

H. Sensitivity Analysis

TABLE XXIII
Sensitivity to Hyperparameters

Parameter	Range Tested	tok/$ Range	Sensitivity
Micro learning rate	$[10^{-5}, 10^{-3}]$	7,800–8,509	Medium
Macro discount factor	$[0.95, 0.999]$	8,100–8,600	Low
SLO Lagrange multiplier	$[0.1, 10]$	7,200–8,820	High
Options termination β	$[0.01, 0.5]$	7,900–8,509	Low
Feudal reward alignment η	$[0.01, 1.0]$	7,400–8,509	Medium

The SLO Lagrange multiplier is the most sensitive parameter, as expected—it directly trades off optimization aggressiveness against constraint satisfaction. We recommend starting with $\lambda = 1.0$ and allowing the dual gradient to adapt.

I. TCO Analysis

TABLE XXIV
Annualized TCO Impact (128-GPU Cluster, 60-Day Extrapolation)

Component	Heuristic	Hier-RL	Savings
Cloud compute	$3.2M	$2.15M	$1.05M
Energy (on-prem)	$890K	$739K	$151K
SLO penalties	$420K	$87K	$333K
Operations labor	$600K	$480K	$120K
RL training compute	—	$45K	—
Total	$5.11M	$3.50M	$1.61M (31%)

J. Computational Overhead Analysis

TABLE XXV
RL System Computational Overhead

Component	CPU	Memory	Latency	GPU
Micro-agent inference	0.1 core	50 MB	0.08 ms	None
Meso-agent inference	0.05 core	30 MB	0.15 ms	None
Macro-agent inference	0.02 core	80 MB	0.5 ms	None
Meta-controller	0.01 core	20 MB	0.1 ms	None
State collection	0.5 core	200 MB	N/A	None
Total overhead	0.68 core	380 MB	0.93 ms	None

The RL system adds <1 CPU core and <400 MB memory overhead per cluster, with <1 ms total decision latency. Critically, no GPU resources are consumed—all agent inference runs on CPU. The agents require 45 GPU-hours for training (one-time cost, amortized over months of deployment).

K. Deployment Lessons Learned

After 60 days of production deployment, we document key operational insights:

Lesson 1 – Trust building is critical: Operators initially distrusted RL decisions, overriding 35% of recommendations in week 1. By week 8, the override rate dropped to 4% as operators observed consistent improvements. The

transparency dashboard—showing RL reasoning and expected outcomes—was the most important trust-building feature.

Lesson 2 – Non-stationarity is the primary challenge: Three non-stationarity events occurred during deployment: (a) DeepSeek-V3 deployment (week 3), requiring micro and meso agent retraining; (b) a provider pricing change (week 5), handled by the macro agent within 1 hour; (c) a traffic pattern shift due to a new enterprise customer (week 7), requiring macro agent retraining. Warm-starting reduced retraining time from 60 days to 2–5 days for all three events.

Lesson 3 – Safety constraints prevent over-optimization: Without constraints, the unconstrained RL achieves 3.7% higher cost-efficiency (Table XX) but drops SLO compliance to 87.3%—unacceptable in production. The Lagrangian relaxation approach provides the right trade-off, and operators appreciate being able to adjust the SLO target without retraining the policy.

Lesson 4 – The meta-controller is worth its complexity: Independent agents (without meta-controller) are simpler to deploy and debug but lose 14.9% efficiency (Table XX). The primary failure mode without coordination: the macro agent routes traffic to a low-cost provider while the meso agent on that provider is in power-save mode, causing latency spikes. The meta-controller prevents such conflicts.

TABLE XXVI
Deployment Timeline and Key Events

Week	Phase	Key Event
1–2	Shadow mode	96% safe, 82% improve
3–4	Canary (5%)	DeepSeek-V3 deployed
5–6	Canary (20%)	Provider pricing change
7–8	Full rollout	New enterprise customer
9–12	Steady state	Override rate: 4%

L. Comparison with Production RL Systems

TABLE XXVII
Comparison with Existing Production RL Systems

System	Domain	Levels	Constraints	Multi-obj	Production
Decima [3]	Scheduling	1	No	No	Sim only
AlphaChip [4]	Chip design	1	Soft	No	Yes
DC cooling [9]	HVAC	1	Soft	No	Yes
Borg RL	Scheduling	1	Hard	No	Yes
Ours	Inference	3+meta	Hard+Soft	4-obj	Yes

XIII. Related Work

A. RL for Systems Optimization

Decima [3] applies RL to cluster job scheduling, achieving 21% better average job completion time. AlphaChip [4] uses RL for chip placement. Both are single-level, single-objective systems. Our work extends to hierarchical, multi-objective, multi-timescale optimization with formal convergence guarantees.

B. Hierarchical RL

The options framework [1] provides temporal abstraction for multi-timescale decisions. Feudal Networks [2] decompose rewards across hierarchy levels. We combine both and provide the first application to infrastructure management, where the natural hierarchy (request→node→fleet) maps directly to the agent hierarchy.

C. Multi-Objective RL

Multi-objective RL has been approached through linear scalarization [?], Pareto optimization [?], and constraint-based methods. Our approach combines all three: the meta-controller uses adaptive linear scalarization (dynamically adjusting weights based on SLO compliance), the per-level agents use constraint-based methods (Lagrangian relaxation for soft SLO constraints, action masking for hard safety constraints), and the fleet-level evaluation uses Pareto analysis to characterize the achievable trade-off frontier.

This hybrid approach is necessary because different objectives require different treatment: throughput and cost can be traded off continuously (scalarization), while SLO compliance has a hard threshold (constraint), and the relationship between energy and performance depends on the current operating regime (Pareto analysis reveals the shape of this relationship).

D. Safe RL and Constrained Optimization

CPO [7] and RCPO [8] enforce constraints during policy optimization. Our extension combines Lagrangian relaxation for soft constraints (latency SLOs) with action masking for hard constraints (thermal limits, OOM prevention), providing both probabilistic and deterministic safety guarantees in a unified framework.

E. Offline RL for Infrastructure

CQL [5] and Decision Transformer [6] enable learning from logged data. We extend offline RL to the hierarchical setting with level-specific conservative penalties that account for the different risk profiles at each timescale—micro-level exploration is cheap (single request), macro-level exploration is expensive (fleet-wide routing changes).

XIV. Discussion and Conclusion

Principal findings: (1) Hierarchical RL achieves 47% better cost-efficiency than heuristics and 31% better than single-level RL. (2) The meta-controller is critical—independent agents without coordination lose 14.9% efficiency. (3) Constrained MDPs maintain 98.8% SLO compliance while optimizing cost. (4) Cross-cloud disaggregated routing (discovered by the Macro-Agent) provides the single largest cost improvement (-23%). (5) RL-based KV-cache eviction outperforms LRU by 14% in cache hit rate for agentic workloads. (6) Convergence is guaranteed under feudal reward alignment with two-timescale updates.

Limitations: (1) Training requires 60+ days of production data per infrastructure configuration. (2) Non-stationarity (new models, hardware changes) requires periodic retraining, though warm-starting reduces adaptation to 2–5 days. (3) The options framework adds 2–5ms decision overhead at the meta-controller level. (4) The Lyapunov convergence guarantee is for local optima only; global optimality is not guaranteed due to non-convexity of the joint policy space. (5) Current implementation requires PyTorch and custom CUDA kernels for the micro-agent, limiting portability.

Future work: (1) Foundation models for RL policies that transfer across hardware configurations without retraining. (2) Multi-agent extensions for federated inference across independent operators. (3) Integration with LLM-based policy explanation for operator trust building. (4) Extension to training workloads (not just inference) where the optimization landscape differs fundamentally.

Conclusion:

A. Failure Mode Analysis

We document the failure modes observed during the 60-day deployment:

TABLE XXVIII
Observed Failure Modes and Recovery

Failure Mode & Frequency & Detection & Impact & Recovery
Micro-agent OOD state & 3/month & KL monitor & None & Auto-fallback
Meso-agent slow update & 1/month & Reward drift & Minor & Re-sync
Macro spot preemption & 8/month & Instant & None & Re-route
Meta goal oscillation & 2/month & Stability check & Minor & Damping
Full system fallback & 0/60 days & N/A & N/A & N/A

The most common failure mode is spot instance preemption (8/month), which is handled seamlessly by the macro-agent re-routing affected traffic within 30 seconds. No full system fallback (all agents to heuristics simultaneously) occurred during the entire 60-day deployment.

B. Scalability Projections

TABLE XXIX
Projected Performance at Larger Scales

Scale & GPUs & Providers & Training & Expected Savings
Current (eval) & 128 & 6 & 45 GPU-hr & 31%
Medium & 512 & 8 & 90 GPU-hr & 34%
Large & 2,048 & 10 & 180 GPU-hr & 38%
Hyperscale & 10,000+ & 12+ & 400 GPU-hr & 42%+

Savings increase with scale because: (1) more providers enable finer-grained cost arbitrage; (2) larger fleets provide more data for training, improving policy quality; (3) cross-cloud routing opportunities increase combinatorially with provider count. The training cost scales sub-linearly due to the hierarchical architecture: only the macro-agent grows with fleet size.

C. Societal Impact

Hierarchical RL for inference optimization has significant implications for the AI industry:

Democratization: By reducing inference costs by 31–42%, this work lowers the barrier to deploying large language models, potentially enabling smaller organizations to compete with hyperscalers.

Sustainability: The 17% energy reduction at current scale would translate to 2.8 GWh annual savings at hyperscale (10K GPUs)—equivalent to removing 430 cars from the road. As AI inference demand grows exponentially, such efficiency gains are critical for sustainability.

Open challenges: Several challenges remain for broader adoption: (1) the 60-day cold-start training requirement is too long for fast-moving startups; (2) the hierarchical architecture adds debugging complexity when issues arise; (3) the options framework requires careful tuning of termination conditions to avoid degenerate behaviors (options that never terminate or always terminate immediately).

D. Implementation Details

TABLE XXX
Implementation Stack and Dependencies

Component	Implementation	Lines of Code
Micro-agent	stable-baselines3 + custom	2,800
Meso-agent	CleanRL + custom	1,900
Macro-agent	Custom options framework	3,200
Meta-controller	Custom PyTorch	1,100
vLLM integration	Custom hooks	530
Safety layer	Custom constraint engine	850
Monitoring	Prometheus + custom	1,200
Dashboard	Grafana + custom panels	600
Total		12,180

The system is deployed as a set of Kubernetes pods alongside the vLLM serving infrastructure. The micro-agent runs as a sidecar container on each vLLM pod (0.1 CPU core, 50MB RAM overhead). The meso and macro agents run as centralized services with sub-second RPC communication to the micro-agents.

E. Comparison with Industry Practice

TABLE XXXI
Hierarchical RL vs. Industry Optimization Approaches

Approach	Throughput	Cost	SLO	Adaptivity
Manual tuning (typical)	Baseline	Baseline	92–95%	None
Auto-scaling rules	+5%	hBc0%	93–96%	Reactive
ML-based prediction	+8%	hBc5%	94–97%	Slow
Single-level RL	+12%	hBc0%	95–97%	Medium
Hierarchical RL	+22%	hBc3%	98.8%	Fast

Workforce: Automated optimization reduces the need for infrastructure tuning expertise (currently a scarce and expensive skill). This could shift infrastructure engineering roles from manual tuning to policy design and safety validation.

Broader impact: Hierarchical RL for infrastructure optimization could transform how cloud providers manage resources. Our approach is applicable beyond inference: training workloads, database systems, and network routing all exhibit similar multi-timescale, multi-objective characteristics. The safety framework (constrained MDPs with action masking) is particularly important—it demonstrates that RL can be deployed in production systems with formal safety guarantees, addressing a key barrier to RL adoption in industry.

Reproducibility: Our implementation builds on stable-baselines3 (micro-agent), CleanRL (meso-agent), and a custom options-framework implementation (macro-agent and meta-controller). The vLLM integration requires custom hooks for batch scheduling and KV-cache management (530 lines of Python). Training data format: standard Prometheus time-series with vLLM-specific metric names.

We presented a hierarchical RL framework for production LLM inference spanning seven orders of magnitude in timescale. The combination of micro/meso/macro agents with feudal coordination, constrained MDPs for SLO guarantees, Lyapunov convergence analysis, and multi-objective Pareto optimization establishes RL as a viable replacement for hand-tuned heuristics in production inference infrastructure. The 31% cost-efficiency improvement over single-level RL and 47% over heuristics demonstrates that hierarchical decomposition is essential for complex infrastructure optimization.

XV. Future Work

Several promising research directions emerge from this work:

Model-based RL: Our current agents are model-free, which requires extensive real-world data collection. Incorporating a learned world model (analogous to DreamerV3 for game environments) could reduce the data requirements by 10–100× through imagination-based training. The challenge is learning accurate transition models for the complex, partially-observed datacenter environment.

Multi-tenant optimization: Our current system optimizes for a single organization's workloads. Extending to multi-tenant cloud environments requires game-theoretic formulations where each tenant's agent optimizes for its own SLOs while sharing physical resources. Preliminary analysis suggests that Nash equilibria in this setting can be 15–20% more efficient than independent optimization.

Hardware-software co-design: The current system takes hardware capabilities as fixed and optimizes software decisions. A natural extension is to include hardware configuration in the action space: GPU clock frequencies, memory bandwidth allocation, and even future hardware procurement decisions. This would require extending the macro-agent's timescale to weeks/months.

Foundation model for RL policies: Rather than training separate policies for each cluster, a foundation policy pre-trained on diverse infrastructure configurations could be fine-tuned with minimal data for new deployments. Initial experiments with policy distillation across 3 cluster configurations show promising zero-shot transfer (65% of fine-tuned performance), suggesting this direction is viable.

Formal verification: While our Lyapunov analysis provides local stability guarantees, formal verification of safety properties across the full state space remains an open challenge. Techniques from safe RL (barrier functions, reachability analysis) could provide stronger guarantees, at the cost of more conservative policies.

TABLE XXXII
Projected Impact of Future Extensions

Extension	Effort	Expected Gain	Risk
World model integration	6 months	+15% efficiency	Medium
Multi-tenant game theory	9 months	+20% utilization	High
HW-SW co-design	12 months	+10% TCO	Medium
Foundation RL policy	6 months	10× faster deploy	Low
Formal verification	12 months	Stronger guarantees	High

We believe the most impactful near-term extension is the foundation RL policy, which addresses the primary deployment barrier—the 60-day cold-start training period. If successful, this could enable "plug-and-play" RL optimization for inference infrastructure, dramatically lowering the barrier to adoption.

XVI. Conclusion

This paper presents a comprehensive hierarchical reinforcement learning framework for optimizing production LLM inference infrastructure across seven orders of magnitude in decision timescale. Our key contributions are:

1) A three-level agent hierarchy (micro, meso, macro) with a meta-controller, enabling simultaneous optimization of per-request scheduling (milliseconds), per-node resource management (seconds), and fleet-wide capacity planning (minutes to hours).
2) A feudal reward decomposition with Lyapunov stability guarantees, ensuring that hierarchical optimization converges and that cross-level coordination improves upon independent optimization.
3) A constrained MDP formulation combining action masking (hard safety constraints) with Lagrangian relaxation (soft SLO constraints), enabling deployment in production systems with formal safety guarantees.
4) A multi-objective optimization framework that jointly optimizes throughput, cost, energy, and SLO compliance, with a Pareto analysis revealing the achievable trade-off frontier.
5) Extensive production evaluation over 60 days on a 128-GPU multi-cloud deployment, demonstrating 22% throughput improvement, 33% cost reduction, and 98.8% SLO compliance.

The 31% cost-efficiency improvement over single-level RL and 47% over hand-tuned heuristics demonstrates that hierarchical decomposition is not merely beneficial but essential for complex infrastructure optimization. As LLM inference workloads continue to grow exponentially, automated optimization through hierarchical RL will become a competitive necessity for inference providers.

Acknowledgments

The authors thank the anonymous reviewers for their constructive feedback. We gratefully acknowledge computing resources provided by AWS, GCP, OCI, CoreWeave, Lambda Labs, and Crusoe Energy for the multi-cloud evaluation. This work was supported in part by the MIT Energy Initiative and the NSF AI Institute for Advances in Optimization.

References

[1] R. Sutton, D. Precup, and S. Singh, "Between mdps and semi-mdps: Temporal abstraction in rl," AI, 1999.
[2] A. Vezhnevets et al., "Feudal networks for hierarchical rl," ICML, 2017.
[3] H. Mao et al., "Learning scheduling algorithms for data processing clusters," SIGCOMM, 2019.
[4] A. Mirhoseini et al., "A graph placement methodology for fast chip design," Nature, 2021.
[5] A. Kumar et al., "Conservative q-learning for offline rl," NeurIPS, 2020.
[6] L. Chen et al., "Decision transformer: Rl via sequence modeling," NeurIPS, 2021.
[7] J. Achiam et al., "Constrained policy optimization," ICML, 2017.
[8] C. Tessler, D. Mankowitz, and S. Mannor, "Reward constrained policy optimization," ICLR, 2018.
[9] S. Levine et al., "Offline rl: Tutorial, review, and perspectives," arXiv:2005.01643, 2020.
[10] W. Kwon et al., "Efficient memory management for llm serving with pagedattention," SOSP, 2023.
[11] P. Patel et al., "Splitwise: Efficient generative llm inference using phase splitting," ISCA, 2024.